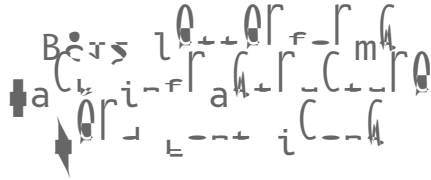


Planetaire Mono

[illegible]

1. The first part of the document is a letter from the author to the reader, dated 1968. The letter is written in a very informal, handwritten style, and is addressed to "Dear Sir". The author is a young man, and he is writing to the reader to inform him of the author's plans to publish a book. The author is very excited about the book, and he is asking the reader to help him in his efforts. The author is also asking the reader to let him know if the reader has any suggestions or comments. The letter is signed "Yours truly, [Name]".

03-lar (200)
04 (200)
04 (700)
EXA 04 (200)
EXA 04 IAU SC (200)

[illegible]

Рэспубліка Беларусь

la-ai r- at a- i- a, h- w- B- s- i- i- l- r- m-
- r- i- m- r- a- a- i- i- a- m- a- l- l- i- a- a- l- w- r- a- l- i- i- a- l- a- y-.

l- i- c- l- r- w- f- x- a- m- r- a- r- l- a- y- t- a- c-
m- y- l- x- w- i- t- f- i- e- j- - - - l- i- r- i- a- . l- w- v- x- i- z- l- y-
l- i- c- a- f- - a- l- a- i- m- r- !

W- t- a- r- a- i- w- o- a- r- a, i- a- r- m- a- m- a- r- a-
W- i- t- r- . e- r- y- y- r- m- a- , l- a- m- i- - - - t- a- i- i- o-
i- i- i- c- f- r- m- t- a- o, t- a- m- a- l- r- l- a- y- a- r- a- r- o- m- .
l- w- a- l- a- r- a- B- s- a- l- m- i- a- r- o- m- .
o- i- i- a- l- l- y- r- a- i- r- c- i- i- r- l- a- y- a- t- w- a- r- a-
m- i- a- a- a- a- c- i- o- r- c- l- m- a- t- a- i- f- f- o- e- o- w- o- ,
e- i- o- . l- a- y- a- i- r- a- i- t- a- r- a- r- a- c- i- i- r- a- c- a- r- a- i- t- a- i- t-
t- a- f- l- i- y- m- l- c- a- a- a- e- i- r- a- m- m- y- - - - : l- a- c- a- r- a- i- t- a- i- t- ,
r- i- r- , a- r- w- , a- r- s- , o- o- i- - - a- y- f- r- a- m- a- r- a- i- t- a- i- t- .

• • • •

r- a- i- r- a- i- a- l- a- i- r- a- r- a- l- a- m- a- r- a- r- a- y- r- a- y- . e- a- l- a- c- a-
a- r- a- x- l- r- m- i- a- l- a- l- a- r- a- y- r- a- y- . e- l- a- l- -
w- r- c- i- l- a- i- i- c- m- i- a- f- o- l- i- a- i- l- l- y- i- i- . i- i- a- m- a- l- i- a- a-
y- a- g- i- s- f- a- l- a- a- a- a- i- .

ΕΕΕΚ

Ξεσπάζω τὴν ψυχ-φθόρα βδελυγμία.

СЪИГГГГ

Съешь же ещё этих мягких французских булок, да выпей чаю. Широкая электрификация южных губерний даст мощный толчок подъёму сельского хозяйства.

PyTorch vs GPT: The Most Atomic Way to Train and Run Inference for a GPT in Pure, Dependency-Free Python

PyTorch is a deep learning framework that allows you to train and run inference for a GPT in pure, dependency-free Python. This file is the complete algorithm. Everything else is just efficiency.

```
"""
The most atomic way to train and run inference for a GPT in pure, dependency-free Python.
This file is the complete algorithm.
Everything else is just efficiency.

@karpathy
"""

import os # os.path.exists
import math # math.log, math.exp
import random # random.seed, random.choices, random.gauss, random.shuffle
random.seed(42) # Let there be order among chaos

# Let there be a Dataset 'docs': List[str] of documents (e.g. a list of names)
if not os.path.exists('input.txt'):
    import urllib.request
    names_url = 'https://raw.githubusercontent.com/karpathy/makemore/988aa59/names.txt'
    urllib.request.urlretrieve(names_url, 'input.txt')
docs = [line.strip() for line in open('input.txt') if line.strip()]
random.shuffle(docs)
print(f'num docs: {len(docs)}')

# Let there be a Tokenizer to translate strings to sequences of integers ("tokens") and back
uchars = sorted(set(''.join(docs))) # unique characters in the dataset become token ids 0..n-1
BOS = len(uchars) # token id for a special Beginning of Sequence (BOS) token
vocab_size = len(uchars) + 1 # total number of unique tokens, +1 is for BOS
print(f'vocab size: {vocab_size}')

# Let there be Autograd to recursively apply the chain rule through a computation graph
class Value:
    __slots__ = ('data', 'grad', '_children', '_local_grads') # Python optimization for memory usage

    def __init__(self, data, children=(), local_grads=()):
        self.data = data # scalar value of this node calculated during forward pass
        self.grad = 0 # derivative of the loss w.r.t. this node, calculated in backward pass
        self._children = children # children of this node in the computation graph
        self._local_grads = local_grads # local derivative of this node w.r.t. its children

    def __add__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        return Value(self.data + other.data, (self, other), (1, 1))

    def __mul__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        return Value(self.data * other.data, (self, other), (other.data, self.data))

    def __pow__(self, other): return Value(self.data**other, (self,), (other * self.data**(other-1),))
    def log(self): return Value(math.log(self.data), (self,), (1/self.data,))
    def exp(self): return Value(math.exp(self.data), (self,), (math.exp(self.data),))
    def relu(self): return Value(max(0, self.data), (self,), (float(self.data > 0),))
    def __neg__(self): return self * -1
    def __radd__(self, other): return self + other
    def __sub__(self, other): return self + (-other)
    def __rsub__(self, other): return other + (-self)
    def __rmul__(self, other): return self * other
    def __truediv__(self, other): return self * other**-1
    def __rtruediv__(self, other): return other * self**-1

    def backward(self):
        topo = []
        visited = set()
        def build_topo(v):
            if v not in visited:
                visited.add(v)
                for child in v._children:
                    build_topo(child)
            topo.append(v)
        build_topo(self)
        self.grad = 1
        for v in reversed(topo):
            for child, local_grad in zip(v._children, v._local_grads):
                child.grad += local_grad * v.grad

# Initialize the parameters, to store the knowledge of the model
n_layer = 1 # depth of the transformer neural network (number of layers)
n_embd = 16 # width of the network (embedding dimension)
block_size = 16 # maximum context length of the attention window (note: the longest name is 15 characters)
n_head = 4 # number of attention heads
head_dim = n_embd // n_head # derived dimension of each head
matrix = lambda nout, nin, std=0.08: [[Value(random.gauss(0, std)) for _ in range(nin)] for _ in range(nout)]
state_dict = {'wte': matrix(vocab_size, n_embd), 'wpe': matrix(block_size, n_embd), 'lm_head': matrix(vocab_size, n_embd)}
for i in range(n_layer):
    state_dict[f'layer{i}.attn_wq'] = matrix(n_embd, n_embd)
```

PyTorch is a deep learning framework that allows you to train and run inference for a GPT in pure, dependency-free Python. This file is the complete algorithm. Everything else is just efficiency.

```

state_dict[f'layer{i}.attn_wk'] = matrix(n_embd, n_embd)
state_dict[f'layer{i}.attn_wv'] = matrix(n_embd, n_embd)
state_dict[f'layer{i}.attn_wo'] = matrix(n_embd, n_embd)
state_dict[f'layer{i}.mlp_fc1'] = matrix(4 * n_embd, n_embd)
state_dict[f'layer{i}.mlp_fc2'] = matrix(n_embd, 4 * n_embd)
params = [p for mat in state_dict.values() for row in mat for p in row] # flatten params into a single list[Value]
print(f'num params: {len(params)}')

# Define the model architecture: a function mapping tokens and parameters to logits over what comes next
# Follow GPT-2, blessed among the GPTs, with minor differences: layernorm -> rmsnorm, no biases, GeLU -> ReLU
def linear(x, w):
    return [sum(wi * xi for wi, xi in zip(w, x)) for wo in w]

def softmax(logits):
    max_val = max(val.data for val in logits)
    exps = [(val - max_val).exp() for val in logits]
    total = sum(exps)
    return [e / total for e in exps]

def rmsnorm(x):
    ms = sum(xi * xi for xi in x) / len(x)
    scale = (ms + 1e-5) ** -0.5
    return [xi * scale for xi in x]

def gpt(token_id, pos_id, keys, values):
    tok_emb = state_dict['wte'][token_id] # token embedding
    pos_emb = state_dict['wpe'][pos_id] # position embedding
    x = [t + p for t, p in zip(tok_emb, pos_emb)] # joint token and position embedding
    x = rmsnorm(x) # note: not redundant due to backward pass via the residual connection

    for li in range(n_layer):
        # 1) Multi-head Attention block
        x_residual = x
        x = rmsnorm(x)
        q = linear(x, state_dict[f'layer{li}.attn_wq'])
        k = linear(x, state_dict[f'layer{li}.attn_wk'])
        v = linear(x, state_dict[f'layer{li}.attn_wv'])
        keys[li].append(k)
        values[li].append(v)
        x_attn = []
        for h in range(n_head):
            hs = h * head_dim
            q_h = q[hs:hs+head_dim]
            k_h = [ki[hs:hs+head_dim] for ki in keys[li]]
            v_h = [vi[hs:hs+head_dim] for vi in values[li]]
            attn_logits = [sum(q_h[j] * k_h[t][j] for j in range(head_dim)) / head_dim**0.5 for t in range(len(k_h))]
            attn_weights = softmax(attn_logits)
            head_out = [sum(attn_weights[t] * v_h[t][j] for t in range(len(v_h))) for j in range(head_dim)]
            x_attn.extend(head_out)
        x = linear(x_attn, state_dict[f'layer{li}.attn_wo'])
        x = [a + b for a, b in zip(x, x_residual)]
        # 2) MLP block
        x_residual = x
        x = rmsnorm(x)
        x = linear(x, state_dict[f'layer{li}.mlp_fc1'])
        x = [xi.relu() for xi in x]
        x = linear(x, state_dict[f'layer{li}.mlp_fc2'])
        x = [a + b for a, b in zip(x, x_residual)]

    logits = linear(x, state_dict['lm_head'])
    return logits

# Let there be Adam, the blessed optimizer and its buffers
learning_rate, beta1, beta2, eps_adam = 0.01, 0.85, 0.99, 1e-8
m = [0.0] * len(params) # first moment buffer
v = [0.0] * len(params) # second moment buffer

# Repeat in sequence
num_steps = 1000 # number of training steps
for step in range(num_steps):
    # Take single document, tokenize it, surround it with BOS special token on both sides
    doc = docs[step % len(docs)]
    tokens = [BOS] + [uchars.index(ch) for ch in doc] + [BOS]
    n = min(block_size, len(tokens) - 1)

    # Forward the token sequence through the model, building up the computation graph all the way to the loss
    keys, values = [[] for _ in range(n_layer)], [[] for _ in range(n_layer)]
    losses = []
    for pos_id in range(n):
        token_id, target_id = tokens[pos_id], tokens[pos_id + 1]
        logits = gpt(token_id, pos_id, keys, values)
        probs = softmax(logits)
        loss_t = -probs[target_id].log()
        losses.append(loss_t)
    loss = (1 / n) * sum(losses) # final average loss over the document sequence. May yours be low.

    # Backward the loss, calculating the gradients with respect to all model parameters
    loss.backward()

    # Adam optimizer update: update the model parameters based on the corresponding gradients
    lr_t = learning_rate * (1 - step / num_steps) # linear learning rate decay
    for i, p in enumerate(params):
        m[i] = beta1 * m[i] + (1 - beta1) * p.grad
        v[i] = beta2 * v[i] + (1 - beta2) * p.grad ** 2
        m_hat = m[i] / (1 - beta1 ** (step + 1))
        v_hat = v[i] / (1 - beta2 ** (step + 1))

```

```

p.data -= lr_t * m_hat / (v_hat ** 0.5 + eps_adam)
p.grad = 0

print(f"step {step+1:4d} / {num_steps:4d} | loss {loss.data:.4f}", end='\r')

# Inference: may the model babble back to us
temperature = 0.5 # in (0, 1], control the "creativity" of generated text, low to high
print("\n--- inference (new, hallucinated names) ---")
for sample_idx in range(20):
    keys, values = [[] for _ in range(n_layer)], [[] for _ in range(n_layer)]
    token_id = BOS
    sample = []
    for pos_id in range(block_size):
        logits = gpt(token_id, pos_id, keys, values)
        probs = softmax([l / temperature for l in logits])
        token_id = random.choices(range(vocab_size), weights=[p.data for p in probs])[0]
        if token_id == BOS:
            break
        sample.append(uchars[token_id])
    print(f"sample {sample_idx+1:2d}: {''.join(sample)}")

```


РҮҢӨТҮРӨ СҮРҮЦӨРӨ

БҮҮС ГҮҮС НҮҮС: РҮ-м БҮС

♦ B C D E F G H I J K L M N O P Q R S T U V W X Y Z

БҮҮС ГҮҮС ГҮҮС: РҮ-м БҮС

a b c d e f g h i j k l m n o p q r s t u v w x y z

ОХИЛ: О-д РҮ-м БҮС (А-а, Б-б, В-в, Г-г, Д-д, Е-е, З-з, И-и, К-к, Л-л, М-м, Н-н, О-о, П-п, Р-р, С-с, Т-т, У-у, Ф-ф, Х-х, Ц-ц, Ч-ч, Ш-ш, Щ-щ, Ъ-ь, Ы-ы, Э-э, Ю-ю, Я-я)

0 1 2 3 4 5 6 7 8 9

БҮҮС ГҮҮС: РҮ-м БҮС

! " # \$ % & ' () * + , - . / : ; < = > ? @ [\] ^ _ ` { | } ~

EXTRA: РҮ-м БҮС

À Á Â Ã Ä Å Æ Ç È É Ê Ë Ì Í Î Ï Ð Ñ Ò Ó Ô Õ Ö Ø Ù Ú Û Ü Ý Þ ß à á â ã ä å æ ç è é ê ë ì í î ï ð ñ ò ó ô õ ö ø ù ú û ü ý þ ß

Ā ā Ă ă Ą ą Ć ć Ĉ ĉ Ċ ċ Č č Ď ě Đ đ Ē ē Ĕ ĕ Ė ė Ę ę Ğ ğ

Ğ ğ Ġ ġ Ģ ģ Ĥ ĥ Ħ ħ Ĩ ĩ Ī ī Ĭ ĭ Ĳ ĳ Ĵ ĵ Ķ ķ

СҮҮС: РҮ-м БҮС

♦ B C D E F G H I J K L M N O P Q R S T U V W X Y Z

α β γ δ ε ζ η θ ι κ λ μ ν ξ ο π ρ σ τ υ φ χ ψ ω

СҮҮС: РҮ-м БҮС

АБВГДЕЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ

абвгдежзийклмнопрстуфхцчшщъыьэюя

Printable ASCII Combinations

ASCII (800)

The quick brown fox jumps over the lazy dog
0123456789 ABCDEFGHIJKLMNOPQRSTUVWXYZ {[(>)]} !@#\$\$%

ASCII (800)

The quick brown fox jumps over the lazy dog
0123456789 ABCDEFGHIJKLMNOPQRSTUVWXYZ {[(>)]} !@#\$\$%

BORD (700)

The quick brown fox jumps over the lazy dog
0123456789 ABCDEFGHIJKLMNOPQRSTUVWXYZ {[(>)]} !@#\$\$%

BORD ASCII (700)

The quick brown fox jumps over the lazy dog
0123456789 ABCDEFGHIJKLMNOPQRSTUVWXYZ {[(>)]} !@#\$\$%

EX1BORD (800)

The quick brown fox jumps over the lazy dog
0123456789 ABCDEFGHIJKLMNOPQRSTUVWXYZ {[(>)]} !@#\$\$%

EX1BORD ASCII (800)

The quick brown fox jumps over the lazy dog
0123456789 ABCDEFGHIJKLMNOPQRSTUVWXYZ {[(>)]} !@#\$\$%

SIZE COMBISZ: ASCII 1 / 1000000 SIZE

The quick brown fox jumps over the lazy dog
The quick brown fox jumps over the lazy dog
The quick brown fox jumps over the lazy dog
The quick brown fox jumps over the lazy dog
The quick brown fox jumps over the lazy dog
The quick brown fox jumps over the lazy dog

Рәһмәт җәһәд рәһмәт

Сәһәбәтләгә бирәләр

I l 1 | -гә әһәдәт . л-әһәдәт л . әһәдәт . гә әһәдәт

0 0 o -гә әһәдәт o . әһәдәт o . л-әһәдәт -

r n m г . . . m - сәһәдәт әһәдәт i- Бәһәд

5 S 8 B әһәдәт - әһәдәт . әһәдәт . әһәдәт B

2 Z 6 G әһәдәт . әһәдәт . әһәдәт . әһәдәт

; : әһәдәт - әһәдәт - әһәдәт - әһәдәт - әһәдәт - әһәдәт

() { } [] гә әһәдәт . әһәдәт . әһәдәт . әһәдәт

• - - әһәдәт . әһәдәт . әһәдәт . әһәдәт

Сәһәбәтләгә бирәләр (сәһәдәтләгә бирәләр)

сәһәдәт (сәһәдәтләгә бирәләр)

сәһәдәт / сәһәдәт (сәһәдәтләгә бирәләр)

0 o o -

0 o o -

сәһәдәт сәһәдәт сәһәдәт сәһәдәт сәһәдәт сәһәдәт

сәһәдәт сәһәдәт сәһәдәт сәһәдәт сәһәдәт сәһәдәт

Бәһәдәтләгә бирәләр

() { } [] < > « » < >

Бәһәдәтләгә бирәләр

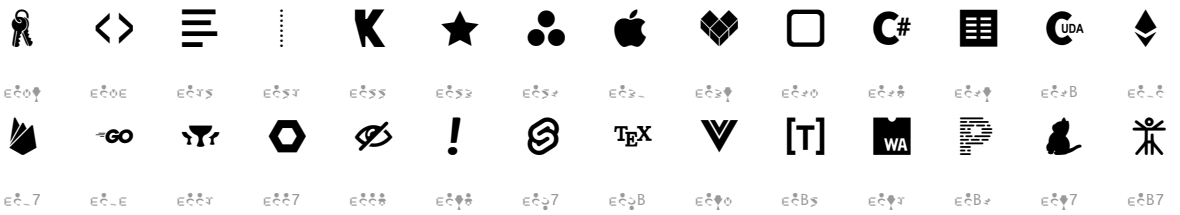
+ - * / = ≠ ≤ ≥ ± × ÷ → ← ↑ ↓

ERG on x conc

$\frac{d}{dt} \left(\int_{\Omega} u^2 dx + \int_{\Gamma} u^2 d\sigma \right) = -2 \int_{\Omega} u \Delta u dx - 2 \int_{\Gamma} u \frac{\partial u}{\partial n} d\sigma$

NI 

LIFE TYPE



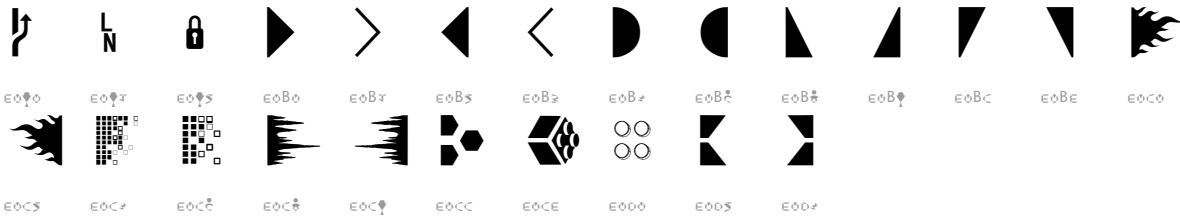
DE/EGOMEDIA



▲◆⊥⊕◆



BOERHIE


$$p_{la^{-0}ai^0} = \frac{1}{2} \left(1 + \frac{C_{m/l}^0}{p_{la^{-0}ai^0}} \right) \cdot 0.675$$

Provenance & Concept

Source code

612 μono	o i d i l o f i l a r t i m i l i l i t i c u r i i l l a y a . l a t a i t a l l a t t (t , a -) , i i i t o o , a n d r a t i / f o o / 6 y i l l i c o l y r t f m b e r s t h e o r d y r f o c i o a c o n t o l c i r c l e t o o i d f l y i g a m i d a t i w i t h c i r c l e (f a l t) a n d r o c t a d l (o r) v a i a t a .
3C	c r i g a m p i n a , y o c o o c i d o f r a r c o c u c r i v i l l a t a r f e n t a r c t r o : f r c t a t i n , y m t l a , a n d v o r a l l m a r i c a .
6 μo n t a	y a o x c r t y r o , i c n r a c i n d f i o c . r s , o o a t o l l r i c i c l i d w o r l i o , e n t w a t m o , o i c t a , m a t e r i a l o o i d , a n d m r o , a l l i n l l a v i a t t a a e n t t a a .

ህዝብ ምክር ቤቱ

Pla-aig x-ri Wil Wira C-m-y-t-rirli-g-i
f-f-t-l-e-r-a-c Wi-t-aia-t-t-rirli-l-a-a-a-c-
a-t-a-c, m-r-a-c B-x-l-l-y-r-n-i-r-a-a, r-am-t
r-a-l, a-r-ar-li-r-r-c-r-i-r-i. E-r-aB-l-w-i-l-a
u-a-r-m B-l-via t-t-r-o-m-t-l-i.

ence

placental and fetal blood samples were obtained from the mother on entrance (OER-1.1).

the over-all welfare, modification, and re-orientation of the
international commercial policy, realizing that modification and
not a total change and carry a different name.

1. C-4 + 1 = 5 Carry 5 Full-Wind 666:

- $\frac{1}{2} \frac{d}{dt} \int_{\mathbb{R}^n} |u|^2 dx = \int_{\mathbb{R}^n} u \Delta u dx = - \int_{\mathbb{R}^n} |\nabla u|^2 dx \leq 0$
- $\frac{1}{2} \frac{d}{dt} \int_{\mathbb{R}^n} |u|^2 dx = \int_{\mathbb{R}^n} u \Delta u dx = - \int_{\mathbb{R}^n} |\nabla u|^2 dx \leq 0$
- $\frac{1}{2} \frac{d}{dt} \int_{\mathbb{R}^n} |u|^2 dx = \int_{\mathbb{R}^n} u \Delta u dx = - \int_{\mathbb{R}^n} |\nabla u|^2 dx \leq 0$